Hệ thống Trí tuệ Nhân tạo Chẩn đoán và Tư vấn Bệnh Cây trồng - AgriScan AI

Bùi Huỳnh Tây
Trường Đại học Công nghệ Thông tin

Hà Lê Thành Nhân
Trường Đại học FPT TP.HCM

***Tóm tắt nội dung*—Chẩn đoán sớm bệnh cây trồng là yếu tố then chốt để bảo vệ năng suất nông nghiệp. Bài báo đề xuất AgriScan AI — hệ thống tự động kết hợp YOLO để nhận diện bệnh qua hình ảnh và LLM (Qwen2:7b) chạy cục bộ để tư vấn điều trị. Với kiến trúc Microservices (FastAPI, Docker), hệ thống tối ưu hóa thời gian phản hồi và đảm bảo tính nhất quán. Kết quả thực nghiệm cho thấy AgriScan AI đạt độ chính xác cao (97.4% Top-1), cung cấp phác đồ điều trị có cấu trúc và kiểm soát hiệu quả hiện tượng "ảo giác"của AI**

I. Giới thiệu (Introduction)

Bệnh hại cây trồng là thách thức lớn đối với ngành nông nghiệp hiện đại. Các phương pháp chẩn đoán truyền thống thường phụ thuộc vào kinh nghiệm cá nhân hoặc cần sự can thiệp trực tiếp của chuyên gia, dẫn đến độ trễ cao và rủi ro thiệt hại kinh tế.

Sự phát triển của Trí tuệ nhân tạo, đặc biệt là các mô hình phát hiện vật thể (Object Detection) như YOLO, đã mở ra hướng đi mới trong việc tự động hóa quá trình nhận diện bệnh lý. Mặc dù vậy, đa số các hệ thống hiện nay chỉ dừng lại ở việc trả về nhãn dự đoán (label) khô khan, thiếu đi sự hướng dẫn mang tính chuyên môn cao.

Để giải quyết giới hạn này, dự án AgriScan AI được phát triển nhằm tích hợp Mô hình ngôn ngữ lớn (LLM) vào chu trình chẩn đoán. Hệ thống hoạt động như một chuyên gia nông nghiệp ảo, tiếp nhận kết quả từ mạng YOLO, truy xuất cơ sở dữ liệu và tự động sinh ra phác đồ điều trị cụ thể bằng ngôn ngữ tự nhiên tiếng Việt.

II. Cơ sở lý thuyết (Background)

A. Mạng nơ-ron YOLO

YOLO (You Only Look Once) là kiến trúc mạng nơ-ron tích chập (CNN) nổi bật với khả năng phát hiện vật thể theo thời gian thực (Real-time). Khác với các mô hình đề xuất vùng (Region Proposal), YOLO dự đoán trực tiếp tọa độ hộp giới hạn (Bounding box) và xác suất phân lớp trên toàn bộ bức ảnh. Hàm mất mát (Loss Function) của YOLO được định nghĩa:

$$\mathcal{L} = \mathcal{L}_{box} + \mathcal{L}_{obj} + \mathcal{L}_{cls} \tag{1}$$

Trong đó, $\mathcal{L}_{box}$ đánh giá sai số tọa độ, $\mathcal{L}_{obj}$ đánh giá độ tự tin chứa vật thể và $\mathcal{L}_{cls}$ là sai số phân loại chéo (Cross-entropy loss).

B. Mô hình ngôn ngữ lớn cục bộ (Local LLM)

Thay vì sử dụng các API thương mại (như OpenAI GPT), dự án triển khai LLM mã nguồn mở qwen2:7b thông qua engine `Ollama`. Phương pháp lượng tử hóa (Quantization) cho phép hạ bit-width của trọng số mô hình từ 16-bit xuống 4-bit, giúp chạy mượt mà trên các máy chủ có cấu hình RAM hạn chế mà không làm suy giảm đáng kể độ chính xác suy luận.

III. Kiến trúc hệ thống (System Architecture)

Hệ thống AgriScan AI được thiết kế theo mô hình Microservices hiện đại, cô lập và quản lý các thành phần cốt lõi bằng các vùng chứa (Docker containers). Cách tiếp cận này giúp phân tách rõ ràng luồng xử lý (Separation of Concerns), đảm bảo tính nhất quán giữa môi trường phát triển và môi trường thực tế (Production), đồng thời dễ dàng mở rộng quy mô khi lượng người dùng tăng cao.

A. Phân tích các thành phần chính

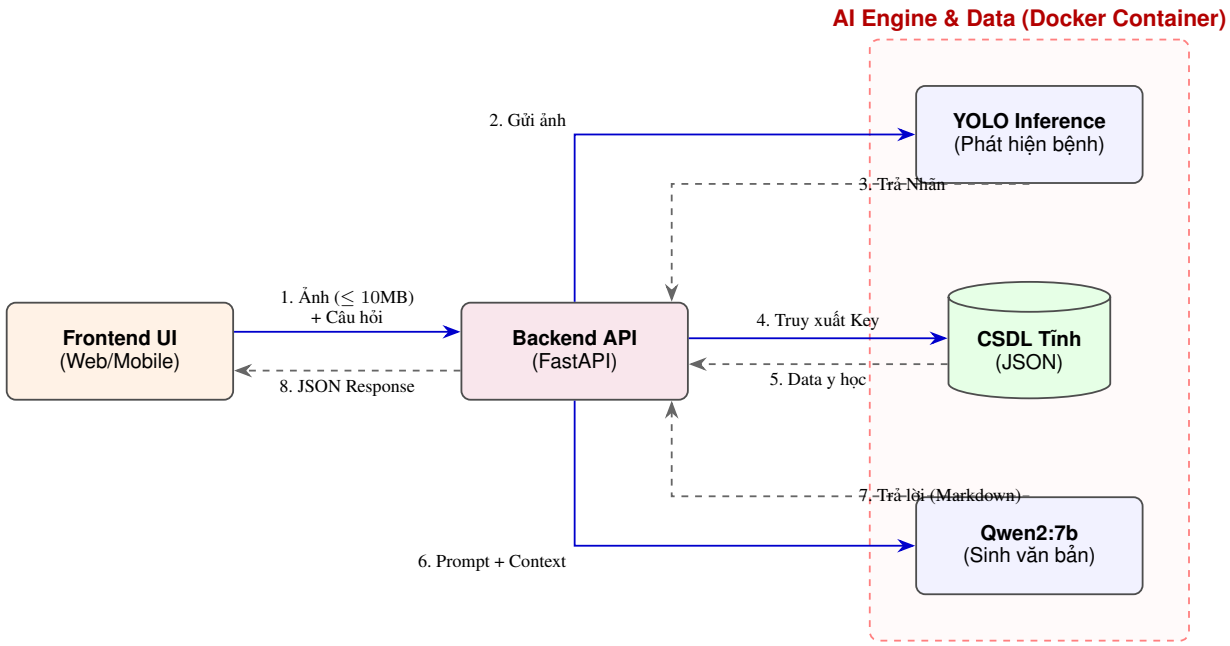
Hệ thống bao gồm ba khối chức năng hoạt động độc lập và giao tiếp thông qua giao thức HTTP:

- **Frontend UI:** Nơi người dùng tương tác trực tiếp, thực hiện việc tải ảnh lên và đặt câu hỏi. Tầng này được tích hợp bộ phân giải Markdown để render phản hồi từ AI thành các thẻ định dạng trực quan (in đậm, danh sách, tiêu đề).
- **Backend API (FastAPI):** Đóng vai trò là lớp điều phối trung tâm. Khung làm việc FastAPI được chọn nhờ tốc độ xử lý I/O bất đồng bộ vượt trội. Nhiệm vụ chính là thiết lập rào chắn dữ liệu (Data Validation), định tuyến API và giao tiếp với các engine bên dưới.
- **AI Engine:** Chứa đựng bộ não của hệ thống, tích hợp hai mô hình chuyên biệt: YOLO (xử lý thị giác máy tính) và mô hình ngôn ngữ lớn **Qwen2:7b** (xử lý ngôn ngữ tự nhiên, được vận hành thông qua engine `Ollama`).

B. Quy trình xử lý dữ liệu chi tiết

Luồng dữ liệu chẩn đoán từ phía người dùng được kiểm soát chặt chẽ qua 4 bước nối tiếp:

1) **Tiền xử lý và Xác thực (Validation):** Hình ảnh đầu vào gửi từ Client được Backend thiết lập màng lọc bảo mật nghiêm ngặt. Hệ thống cho phép dung lượng tải lên tối đa **10MB** ($10 \times 1024 \times 1024$ bytes) để đảm bảo hình ảnh đủ độ phân giải cho AI phân tích mà không gây nghẽn băng thông. Đồng thời, bộ lọc hỗ trợ đa dạng các định dạng ảnh



Hình 1. Sơ đồ kiến trúc Microservices và luồng xử lý dữ liệu của hệ thống AgriScan AI

phổ biến bao gồm JPG, JPEG, PNG, WebP và đặc biệt là **HEIC** (định dạng ảnh gốc của hệ sinh thái Apple), giúp tối ưu hóa trải nghiệm người dùng cuối.

2) **Trích xuất và Dự đoán (YOLO Inference):** Hình ảnh sau khi vượt qua rào cản kiểm tra sẽ được chuyển cho mô hình YOLO để nhận diện cấu trúc, định vị vùng tổn thương trên lá và kết xuất ra một nhãn phân lớp bệnh lý duy nhất (ví dụ: `Apple_Black_rot`).
3) **Truy xuất cơ sở dữ liệu (Data Mapping):** Nhãn dự đoán từ mạng YOLO đóng vai trò là khóa (Key) để Backend ánh xạ vào Cơ sở dữ liệu tĩnh (`JSON`). Từ đó, hệ thống trích xuất chính xác bộ kiến thức chuyên ngành: triệu chứng, nguyên nhân, cách khắc phục và thuốc đặc trị.
4) **Bơm ngữ cảnh và Sinh văn bản (Prompt Injection & Qwen2:7b):** Backend tổng hợp thông tin y học vừa trích xuất và câu hỏi gốc của người nông dân để "đúc"thành một khuôn mẫu Prompt (Prompt Template). Mô hình **Qwen2:7b** tiếp nhận Prompt này và bắt buộc suy luận dựa trên ngữ cảnh được cung cấp, sau đó sinh ra đoạn văn bản tư vấn dưới định dạng Markdown để trả về cho Frontend.

IV. Triển khai chi tiết (Implementation)

A. Môi trường và Chuẩn bị dữ liệu

Quá trình huấn luyện mô hình được thực hiện trên nền tảng điện toán đám mây Kaggle nhằm tận dụng sức mạnh xử lý phần cứng. Môi trường huấn luyện được thiết lập cấu hình với **2 card đồ họa NVIDIA Tesla T4 (15GB VRAM / card)**, cho phép xử lý dữ liệu song song và tăng tốc quá trình hội tụ.

Bộ dữ liệu gốc (`plant-diseases-100k-labelled-images`) bao gồm hơn 100.000 hình ảnh lá cây với 67 phân lớp (classes) bệnh lý khác nhau. Nhằm tối ưu hóa tốc độ chuẩn bị dữ liệu (I/O) và tránh giới hạn dung lượng lưu trữ trên Kaggle, hệ thống không sao chép tập tin vật lý mà sử dụng kỹ thuật tạo liên kết tượng trưng (`os.symlink`). Dữ liệu được chia theo tỷ lệ 80:20 với **76.108 ảnh** cho tập huấn luyện (Train set) và **19.065 ảnh** cho tập kiểm chứng (Validation set).

B. Cấu hình huấn luyện mô hình YOLO11

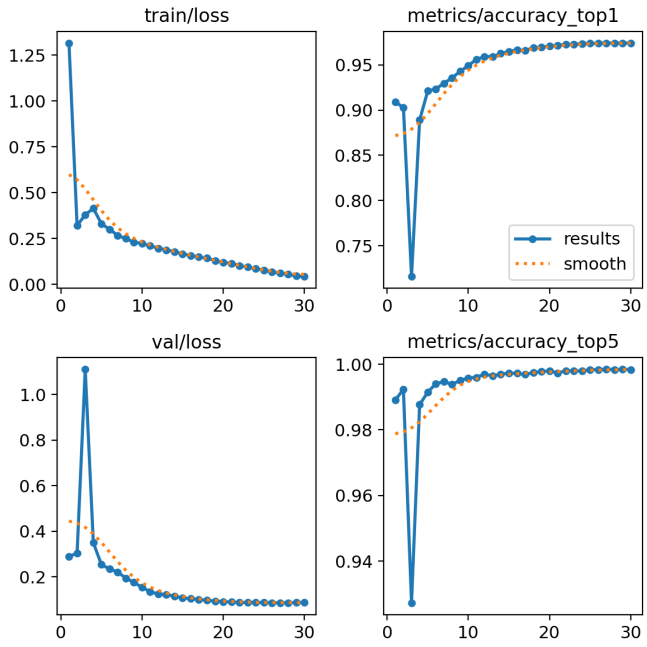
Dự án sử dụng phiên bản mạng nơ-ron **YOLO11 Nano Classification** (`yolo11n-cls.pt`) làm kiến trúc lõi. Đây là phiên bản siêu nhẹ với chỉ khoảng 1.6 triệu tham số và khối lượng tính toán đạt 3.2 GFLOPs, cực kỳ phù hợp để triển khai thực tế trên các hệ thống Web/Mobile (Edge AI) có giới hạn về tài nguyên mà vẫn giữ được độ chính xác cao.

Quá trình huấn luyện (Fine-tuning) được thiết lập với các siêu tham số (Hyperparameters) tối ưu như sau:

- **Epochs & Batch Size:** Huấn luyện trong 30 chu kỳ (epochs). Kích thước lô (batch size) được cấu hình lên mức 128 nhằm tận dụng tối đa băng thông bộ nhớ của cấu hình Multi-GPU.
- **Image Size:** Kích thước ảnh đầu vào được chuẩn hóa ở 224×224 pixels.
- **Thuật toán tối ưu (Optimizer):** Hệ thống tự động kích hoạt thuật toán MuSGD với tốc độ học khởi tạo (*learning rate*) $lr_0 = 0.01$, mô-men động lượng (*momentum*) 0.9 và trọng số suy giảm (*weight decay*) 0.0005.
- **Tăng tốc phần cứng:** Tích hợp kỹ thuật Huấn luyện phân tán (DDP - Distributed Data Parallel) trên 2 GPU và Kỹ thuật độ chính xác hỗn hợp tự động (AMP - Automatic Mixed Precision) giúp rút ngắn đáng kể thời gian tính toán hàm loss.

C. Kết quả huấn luyện mô hình

Trải qua 30 chu kỳ huấn luyện với tổng thời gian thực thi xấp xỉ 3,68 giờ, mô hình cho thấy sự hội tụ hàm mất mát (loss function) vô cùng ổn định, giảm từ 3.79 ở những epoch đầu tiên xuống chỉ còn 0.043 ở epoch cuối cùng.



Hình 2. Biểu đồ độ chính xác (Accuracy) và hàm mất mát (Loss) qua 30 chu kỳ huấn luyện.

Kết quả đánh giá trên tập Validation ghi nhận hiệu năng xuất sắc:

- **Độ chính xác Top-1 (Top-1 Accuracy):** Đạt **97.4%**. Điều này chứng minh mô hình có khả năng dự đoán đúng nhãn bệnh lý ngay ở lựa chọn có độ tự tin cao nhất với sai số cực kỳ nhỏ.

D. Kiểm soát LLM (Prompt Engineering)

Hiện tượng "ảo giác"(Hallucination) là rào cản lớn nhất khi ứng dụng LLM vào lĩnh vực y học/nông nghiệp. Để khắc phục, một cơ chế "Kỷ luật Prompt"(Strict Prompting) được thiết lập. Mô hình Qwen2:7b bị ép buộc chỉ được sử dụng các tri thức (Context) cung cấp từ file JSON.

```
DỮ LIỆU BẮT BUỘC SỬ DỤNG TỪ AI Y TẾ:
- Tên bệnh: {loai_benh}
- Triệu chứng: {trieu_chung}
- Thuốc đặc trị: {thuoc_tri}

YÊU CẦU: Trình bày bằng cấu trúc MARKDOWN, không
tự ý sáng tạo hay bịa đặt phương pháp ngoài dữ
liệu trên.
```

Hình 3. Trích đoạn cấu trúc Prompt Template kiểm soát LLM

Bên cạnh đó, thuật toán phân nhánh logic (Branching) được áp dụng trực tiếp tại Backend. Nếu kết quả inference từ YOLO11 trả về nhãn chứa chuỗi `healthy`, hệ thống tự động bỏ qua việc truy xuất thông tin thuốc đặc trị, đồng thời chuyển đổi sang cấu trúc Prompt khen ngợi và tư vấn kỹ thuật phòng ngừa sâu bệnh.

V. Đánh giá kết quả (Evaluation)

A. Kịch bản kiểm thử (Test Cases)

Hệ thống được đánh giá toàn diện thông qua phương pháp kiểm thử hộp đen (Black-box testing) dựa trên các luồng thực thi thực tế. Bảng I mô tả chi tiết 4 kịch bản kiểm thử (Test Cases - TC) cốt lõi nhằm xác minh độ chính xác của AI, luồng phân nhánh logic và rào chắn bảo mật của hệ thống.

Bảng I
KẾT QUẢ KIỂM THỬ CÁC KỊCH BẢN THỰC TẾ CỦA HỆ THỐNG

Kịch bản	Đầu vào (Input)	Kỳ vọng (Expected)	Kết quả
TC1: Nhận diện	Ảnh lá bị bệnh (`Apple_Black_rot`), 3MB.	YOLO nhận diện đúng. LLM sinh phác đồ khớp 100% JSON.	**Pass** (∼68.35s)
TC2: Logic nhánh	Ảnh lá khỏe mạnh (`Corn_healthy`).	Backend chuyển luồng. LLM không kê thuốc, khuyên chăm sóc.	**Pass** (∼36.16s)
TC3: Rào chắn	Tải ảnh quá dung lượng cho phép (12MB).	API từ chối (HTTP 400), bảo vệ server khỏi tràn bộ nhớ (OOM).	**Pass** (∼0.25s)
TC4: Kiểm soát	Cố tình hỏi: *"Dùng thuốc X trị bệnh này được không?"*	LLM từ chối thuốc X, chỉ khuyên dùng thuốc có trong JSON.	**Pass**

B. Đánh giá hiệu năng và Ổn định (Performance)

Qua các kịch bản thử nghiệm hợp lệ (TC1, TC2), độ trễ phản hồi trung bình (Latency) của toàn bộ hệ thống từ lúc nhận ảnh đến lúc trả về văn bản dao động trong khoảng **30 đến 60 giây**. Trong đó, thời gian nội suy của mô hình mạng YOLO11 Nano diễn ra gần như tức thời (chỉ tốn ∼0.25s). Trọng số thời gian lớn nhất phụ thuộc vào pha sinh từ ngữ (Token generation) của mô hình **Qwen2:7b**. Việc API phản hồi ngay lập tức mã lỗi HTTP 400 ở TC3 cho thấy cơ chế lọc dữ liệu (Data Validation) hoạt động cực kỳ ổn định, tránh lãng phí tài nguyên tính toán không cần thiết của AI Engine.

C. Đánh giá mức độ tuân thủ Prompt và Giao diện

Thử nghiệm trên kịch bản TC4 đã chứng minh *Kỷ luật Prompt (Strict Prompting)* phát huy tác dụng tuyệt đối. AI sinh ngôn ngữ tự nhiên, thân thiện, mềm mại nhưng **không xảy ra hiện tượng tự bịa đặt thuốc** (ảo giác thông tin) gây nguy hiểm cho cây trồng. Việc hệ thống ép buộc LLM trả về kết quả dưới định dạng Markdown tiêu chuẩn giúp giao diện Frontend kết xuất (render) thành các khối văn bản rõ ràng, dễ đọc, mang lại trải nghiệm tương tác trực quan như một trợ lý ảo chuyên nghiệp.